МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

Тема: «Поиск с возвратом»

Вариант 3р

Студент гр. 4343 _____ Толмачев И.А.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N - 1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу — квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы — одно целое число N ($2 \leq N \leq 40$).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \leq x$, $y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Исследовать зависимость кол-ва операций от размера квадрата.

Описание алгоритма и оценка его сложности.

Алгоритм решает задачу разбиения квадрата размером N на целочисленные квадраты методом рекурсивного бэктрекинга. На каждом шаге ищется самая верхняя левая свободная клетка и пробуются все возможные квадраты, которые можно поместить, от максимально возможного размера до единичного. Для каждой попытки квадрат условно ставится на доску, обновляются маски строк и счетчик заполненных клеток, рекурсивно вызывается та же процедура для оставшейся части поля, а затем состояние откатывается. По достижении полного покрытия алгоритм сравнивает количество использованных квадратов с текущим лучшим и при улучшении

запоминает решение. Ветви, в которых текущее число квадратов уже не лучше найденного, отсеиваются. Проверки допустимости постановки квадрата выполняются путем побитовой маски для каждой строки, а операции постановки/снятия модифицируют соответствующие битовые маски строк и счетчик заполненных клеток.

Оценка временной сложности: в худшем случае этот алгоритм работает экспоненциально относительно размера квадрата. Когда мы находим пустую клетку, мы можем попробовать поставить туда квадраты разных размеров — от 1 до N, то есть примерно N вариантов. В глубину рекурсия может уйти очень далеко, если мы будем ставить только квадраты 1x1, то их потребуется N^2 штук. Получается, что общее количество вариантов примерно равно N в степени N^2, количество вариантов на каждом шаге возводится в степень глубины. Это огромное число даже для небольших N. Это очень грубая оценка. На практике каждая попытка поставить квадрат требует выполнения нескольких операций: проверки, можно ли его поставить, самого размещения или удаления, а также поиска следующей свободной клетки. Все эти операции занимают время, пропорциональное N или N^2, то есть полиномиальное время. Но главная проблема даже не в этом, а в том, что самих попыток экспоненциально много. Поэтому в итоге мы получаем время работы, которое можно записать как N в степени N^2, умноженное на некоторый многочлен от N. Иными словами, алгоритм работает экспоненциально долго, и для больших N полный перебор становится невозможным без оптимизации и ускоряющих приемов.

Оценка используемой памяти: основное рабочее состояние — массив битовых масок по строкам размером $O(N)$, N слов по 64 бита для $N \leq 40$, и стек рекурсии с хранением текущего решения. В худшем случае текущий набор квадратов может занимать до $O(N^2)$ записей, каждая — три целых числа, а стек рекурсии соответствует глубине числа размещенных квадратов и тоже ограничен $O(N2)$. Итого асимптотически используемая память составляет $O(N^2)$ слов/записей в худшем случае, плюс $O(N)$ для масок, что дает суммарно $O(N^2)$.

Описание способа хранения частичных решений.

Для хранения текущего состояния доски используется битовая маска. Поскольку максимальный размер квадрата в задаче ограничен числом 40, каждая строка доски кодируется 64-битным беззнаковым целым числом *unsigned long long*. Массив *board_mask* из 40 элементов хранит состояние всех строк, если в i-й строке j-й бит установлен в единицу, это означает, что клетка с координатами (i, j) занята каким-то размещенным квадратом. Дополнительно ведется счетчик *filled_cells*, который хранит общее количество занятых клеток.

Текущее частичное решение хранится в векторе *cur*, который содержит кортежи из трех целых чисел: координаты x и y левого верхнего угла квадрата и его размер. Координаты хранятся в формате от 1 до N, хотя внутренние функции работают с индексацией от 0 для удобства побитовых операций. При нахождении нового решения, которое использует меньше квадратов, чем текущее лучшее, содержимое *cur* копируется в вектор *ans*. Это сделано для того, чтобы после завершения поиска можно было вывести не просто количество квадратов, но и их конкретное расположение. Копирование происходит только когда найдено улучшение, то есть не слишком часто, поэтому накладные расходы на него незначительны.

Такая организация хранения обеспечивает хороший баланс между скоростью работы и простотой кода.

Использованные оптимизации алгоритма.

Для ускорения работы алгоритма и возможности обрабатывать квадраты вплоть до 40 было применено несколько ключевых оптимизаций. Первая оптимизация — это отсечение по текущему лучшему *best*, который устраняет все ветви, уже не превосходящие найденный лучший результат. Кроме того, добавлена простая нижняя оценка числа требуемых дополнительных квадратов. Оценивается максимально возможный квадрат, который

теоретически можно поставить в текущей позиции *max_possible = min(N - x, N - y)*, и по площади оставшейся области вычисляется *min_additional = ceil(remaining / (max_possible^2))*. Эта оценка хоть и грубая, но позволяет отсечь очевидно бесперспективные ветви до начала глубокого перебора размеров.

Вторая оптимизация связана с порядком перебора размеров квадратов. Алгоритм всегда находит самую левую верхнюю свободную клетку и пытается размещать квадраты от максимально возможного размера в этой позиции до минимального. Использование крупных квадратов на ранних этапах позволяет быстрее приблизиться к полному заполнению и либо найти хорошее решение с малым числом квадратов, либо достичь значения *best* и отсечь ветви с мелкими квадратами. Если бы перебор начинался с маленьких квадратов, алгоритм бы уходил в глубокие ветви с большим количеством квадратов и только потом пробовал крупные, что значительно замедлило бы поиск.

Третья оптимизация — это добавление явного счетчика *filled_cells*, который инкрементируется при постановке квадрата и декрементируется при снятии, чтобы избежать частого полного пересчета заполненных клеток. Это делает получение текущего числа занятых клеток O(1) и избавляет от обхода всех строк и подсчёта битов в каждом узле дерева поиска — экономия огромная, потому что подсчет заполняемости происходил бы очень часто.

Четвертая оптимизация — это реализация первичной компактности и быстрых проверок через побитовые маски по строкам *board_mask*. Каждая строка хранится в одном *unsigned long long*, бит *j* отражает занятость клетки (*i*, *j*). Это даёт возможность проверять занятость целого отрезка столбцов за одну операцию формирования маски и побитовой проверки *&*, а операции постановки/снятия квадрата выполняются через || и *&&* с той же маской на *size* строк. Таким образом, проверка возможности установки квадрата размера *s* выполняется за O(*s*) операций чтения/записи строк и за константную стоимость побитовых операций вместо поэлементного обхода, что значительно уменьшает константный множитель в проверках и откатах.

Пятая оптимизация - добавление обрезки симметрии. Внутри цикла по размерам ведётся множество *tried_sizes*, чтобы не пробовать одинаковый размер дважды в одной и той же позиции, это защищает от дубликатных вариантов, которые по геометрии одинаковы в рамках выбранной позиции. Это простая локальная симметрия, которая экономит повторяющиеся ветви, возникающие, например, когда несколько путей приводят к одному и тому же набору незаполненных клеток в пределах строки.

Шестая оптимизация, не связанная напрямую с поиском с возвратом, но критически важна для работы программы в целом, — это обработка частных случаев для N, кратных 2, 3 и 5. Для таких N существуют известные оптимальные разбиения, которые можно выдать мгновенно без всякого перебора.

Описание рекурсивной функции.

Рекурсивная функция в программе называется *backtrack* и имеет следующую сигнатуру: *void backtrack(int used)*. Она не возвращает никакого значения, а все результаты работы сохраняются в глобальных переменных *best* и *ans*. Аргумент *used* передаёт количество квадратов, уже использованных в текущем частичном решении к моменту вызова функции. Назначение функции заключается в систематическом переборе всех возможных способов заполнения оставшейся свободной области квадратами. Функция реализует поиск в глубину с возвратом. Она пытается разместить очередной квадрат, затем рекурсивно вызывает себя для заполнения того, что осталось, а после возврата убирает этот квадрат и пробует следующий вариант.

Тестирование.

Тестирование проводилось автоматизированно с помощью набора юниттестов на Google Test. В них проверялись граничные случаи: для $N = 3$, 5, 7, 11 проверялось свойство, что алгоритм возвращает число квадратов не

меньше N; для больших простых N = 19, 29, 37 проверялась адекватность результата, положительное, не превосходит N^2 и отличается от тривиальных малых значений.

Исследование.

В ходе исследования была проведена серия запусков алгоритма для значений N от 2 до 40. Для каждого N фиксировалось количество операций, выполненных алгоритмом, счетчик элементарных действий при поиске решения. Полученные данные были разделены на две группы: простые значения N и составные значения N. Для каждой группы результаты сохранялись в отдельный csv файл, после чего строился общий график зависимости количества операций от размера N с двумя линиями, см. рисунок 1.

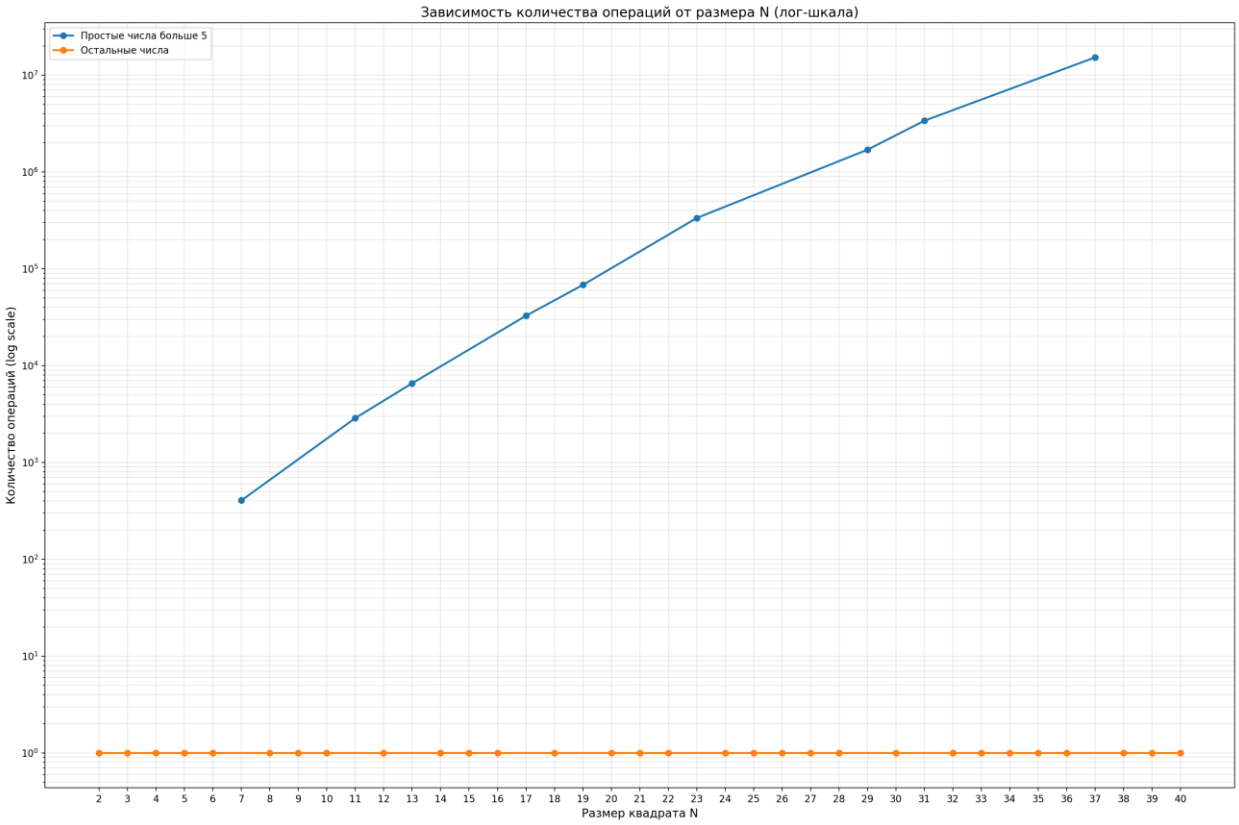

Рисунок 1 — Зависимость количества операций от размера квадрата

По полученному графику видно, что для составных чисел, количество операций 1. Это связано с тем, что в программе предусмотрены конструктивные схемы разбиения для таких случаев.

Для простых чисел наблюдается монотонно возрастающая функция. В этих случаях отсутствуют простые регулярные схемы разбиения, поэтому алгоритм вынужден выполнять более глубокий перебор с отсечениями. Количество рекурсивных вызовов и проверок размещения квадратов существенно возрастает, что отражается в росте значения счетчика операций.

Таким образом, эксперимент подтверждает, что вычислительная сложность задачи определяется не только размером N, но и его структурой как числа. Наличие делителей позволяет алгоритму быстрее найти решение благодаря заранее известным схемам разбиения, тогда как для простых чисел задача требует существенно большего объёма перебора, что приводит к росту количества операций.